

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 — Git & Docker Technical Assignment

**“ATHLETE-SHIELD AI: INTELLIGENT
ATHLETE INJURY RISK PREDICTION &
PERFORMANCE MONITORING”**

Git & Docker Technical Assignment

Submitted in the partial fulfilment for the Course of
CSA1063 (CSA10) — SOFTWARE ENGINEERING

to the award of the degree of
**BACHELOR OF TECHNOLOGY IN
ARTIFICIAL INTELLIGENCE & DATA SCIENCE**

Submitted by
VENNAMPALLI BOOPATHY KHANISHK (192524311)

Under the Supervision of
Dr Anita Davamani K

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences, Chennai – 602105
AUGUST 2026

TABLE OF CONTENTS

1. PROBLEM ANALYSIS	4
1.1 Assigned Problem Statement	4
1.2 Project Objectives	4
1.3 Functional Requirements (Relevant to this Assignment)	4
1.4 Expected Outcomes	4
2. PROJECT STRUCTURE DESIGN.....	5
2.1 Repository Folder Structure.....	5
2.2 Justification of Folder and File Purpose	5
3. GIT REPOSITORY INITIALIZATION	6
3.1 Initializing the Repository.....	6
3.2 Purpose of Essential Git Files	6
4. GIT BRANCHING STRATEGY	7
4.1 Selected Model: Gitflow	7
4.2 Justification	7
5. VERSION CONTROL WORKFLOW	8
5.1 Branch Creation and Feature Development.....	8
5.2 Merging via Pull Request.....	8
5.3 Conflict Resolution	8
5.4 Repository Synchronization.....	8
6. COMMIT HISTORY ANALYSIS	9
6.1 Sample Commit History	9
6.2 Commit Message Conventions	9
7. DOCKER ENVIRONMENT DESIGN	9
7.1 Why Docker Is Suitable for This Project.....	9
7.2 Application Components Requiring Containerization.....	10
8. DOCKERFILE DEVELOPMENT	10
8.1 Dockerfile — Application Server (backend/Dockerfile).....	10
8.2 Dockerfile — Injury Prediction Engine (ml-service/Dockerfile)	10
8.3 Instruction-by-Instruction Explanation	11
9. DOCKER COMPOSE DESIGN	11
9.1 docker-compose.yml	11
9.2 Explanation of Services, Networking, Volumes, and Dependencies.....	12

10. CONTAINER DEPLOYMENT AND TESTING	12
10.1 Building and Running the Stack	12
10.2 Testing Procedure and Evidence.....	13
11. BENEFITS OF GIT AND DOCKER FOR THIS PROJECT	13
12. CHALLENGES AND IMPROVEMENTS	14
12.1 Challenges Encountered.....	14
12.2 Suggested Improvements and Future Enhancements	14
CONCLUSION.....	14
REFERENCES	15

1. PROBLEM ANALYSIS

1.1 Assigned Problem Statement

The assigned project is the Injury Risk Prediction System for Athletes Using Performance Data (branded as Athlete-Shield AI), a system designed to monitor athlete performance metrics and predict the likelihood of injuries before they occur. The system consumes historical and real-time performance data, applies machine learning techniques through a dedicated prediction engine, and returns risk scores and recommendations to coaches and athletes. This report applies Git version control and Docker containerization concepts to the development and deployment of this system, in line with the CO3 Assessment Tool 1 requirement to build full-stack applications with an emphasis on containerization, version control, and collaborative development.

1.2 Project Objectives

- Collect and manage athlete performance data through a secure, multi-user web/mobile portal.
- Analyze training and health-related metrics to identify patterns associated with injury risk.
- Predict injury risks using a dedicated machine learning prediction engine, decoupled from the main application server.
- Generate Low/Medium/High risk scores together with actionable recommendations for coaches.
- Package every component of the system into portable, reproducible containers so that the same environment runs identically on a developer's laptop, a CI server, and production.
- Maintain a clean, auditable Git history so that multiple contributors (frontend, backend, and ML engineers) can collaborate without overwriting one another's work.

1.3 Functional Requirements (Relevant to this Assignment)

Table 1.1: Functional requirements mapped to system modules

ID	Requirement	Owning Module / Container
FR1–FR3	User registration, authentication, athlete profile management	Application Server (Node.js)
FR4–FR5	Performance data upload and historical data storage	Application Server + MySQL Database
FR6	Data cleaning, validation, and transformation	Data Processing Module
FR7–FR8	Injury risk prediction and risk score generation	Injury Prediction Engine (Python/ML)
FR9	Alerts and notifications for high-risk situations	Alert & Reports Module
FR10	Reports and visual dashboards	Web / Mobile Portal (frontend)

1.4 Expected Outcomes

By the end of this assignment, the project is expected to have: a well-organized, version-controlled repository with a documented branching strategy; a set of Dockerfiles that containerize each independently deployable component; a Docker Compose configuration that brings up the full multi-service stack with a single command; and a demonstrated, screenshot-evidenced Git and Docker workflow covering commits, branching, merging, and container deployment.

2. PROJECT STRUCTURE DESIGN

2.1 Repository Folder Structure

The project is organized as a monorepo with clearly separated folders for each independently deployable service, mirroring the layered architecture (Presentation, Application, Data Processing, Prediction Engine, Database, and Alert/Reports layers) established in the earlier architecture design assignment.

```
athlete-shield-ai/
|-- frontend/                # Web / Mobile Portal (React)
|   |-- src/
|   |-- public/
|   |-- package.json
|   |-- Dockerfile
|-- backend/                 # Application Server (Node.js + Express)
|   |-- src/
|   |   |-- routes/          # Athlete, Auth, Reports routes
|   |   |-- controllers/
|   |   |-- middleware/      # Auth, validation
|   |   |-- services/        # Data Processing Module
|   |-- package.json
|   |-- Dockerfile
|-- ml-service/              # Injury Prediction Engine (Python)
|   |-- model/               # Trained model artifacts
|   |-- app.py               # Flask/FastAPI inference service
|   |-- requirements.txt
|   |-- Dockerfile
|-- database/
|   |-- init.sql             # Schema: athletes, training, injuries
|   |-- migrations/
|-- docs/
|   |-- architecture-diagram.png
|   |-- workflow-diagram.png
|-- .github/                 # CI workflow (future enhancement)
|-- docker-compose.yml
|-- .gitignore
|-- .env.example
|-- README.md
```

2.2 Justification of Folder and File Purpose

Table 2.1: Purpose of major repository components

Folder / File	Purpose
frontend/	Isolates all UI code so the web/mobile portal can be built and deployed independently

Folder / File	Purpose
backend/	Houses the application server; routes, controllers, and services map directly to the layered architecture
ml-service/	Keeps the injury prediction engine's Python dependencies fully separate from the Node.js backend
database/init.sql	Defines the MySQL schema so any developer can reproduce the database from scratch
docker-compose.yml	Single source of truth for how all services are networked and started together
.gitignore	Prevents node_modules, __pycache__, .env, and build artifacts from polluting the repository
.env.example	Documents required environment variables without committing real secrets
README.md	Onboarding document explaining setup, architecture, and contribution guidelines

3. GIT REPOSITORY INITIALIZATION

3.1 Initializing the Repository

The repository was created locally and connected to a remote origin using the following sequence of commands, establishing version control before any application code was written.

```
$ mkdir athlete-shield-ai && cd athlete-shield-ai
$ git init
Initialized empty Git repository in athlete-shield-ai/.git/
$ git branch -M main
$ echo "node_modules/\n__pycache__/\n.env\n*.log" > .gitignore
$ echo "# Athlete-Shield AI" > README.md
$ git add .gitignore README.md
$ git commit -m "chore: initial commit with gitignore and readme"
$ git remote add origin https://github.com/<user>/athlete-shield-ai.git
$ git push -u origin main
```

3.2 Purpose of Essential Git Files

Table 3.1: Essential Git files and their roles

File / Folder	Purpose
.git/	Hidden directory created by git init; stores the entire object database, refs, and history
.gitignore	Tells Git which files/folders to never track (build output, secrets, dependency caches)
README.md	First point of reference for any collaborator; documents setup and usage
.env.example	Template for required environment variables, committed in place of the real .env

4.1 Selected Model: Gitflow

Because the project has three functionally distinct, independently deployable components (frontend, backend, ML service) being developed in parallel, a Gitflow-style branching model was adopted. This model separates long-lived integration branches from short-lived, purpose-specific branches, which keeps the main branch always deployable.

Branch	Purpose
main	Always reflects the latest stable, production-ready release
development	Integration branch where completed features are merged and tested together
feature/frontend-dashboard	Isolated work on the athlete/coach dashboard UI
feature/ml-prediction-engine	Isolated work on the injury risk model and inference API
feature/docker-setup	Isolated work on Dockerfiles and docker-compose configuration
release/v1.0	Final stabilization and version bump before a production release
hotfix/db-connection-fix	Urgent fix branched directly from main for a production issue

Gitflow is appropriate for this project because the three services are owned by conceptually different concerns (UI, business logic, and machine learning) that are frequently developed concurrently but must be integration-tested together before release. Feature branches let each concern be developed and reviewed in isolation without destabilizing development, the development branch provides a shared integration point for testing the full Docker Compose stack, and hotfix branches allow a critical production issue — for example, a broken database connection string in the deployed container — to be resolved without waiting for the next full release cycle.

```

main      -----o-----o-----o-----> (stable releases)
              /         /         /
release/v1.0 o-----' (hotfix merged back)
              /

```

```

development  -----o-----o-----o-----o----->  (integration)
                    /      /      /
feature/frontend-----' /      /
feature/ml-engine-----' /
feature/docker-setup-----'

```

5. VERSION CONTROL WORKFLOW

5.1 Branch Creation and Feature Development

```

$ git checkout development
$ git checkout -b feature/ml-prediction-engine
Switched to a new branch 'feature/ml-prediction-engine'
# ... implement model training and inference code in ml-service/ ...
$ git add ml-service/
$ git commit -m "feat(ml): add injury risk prediction model and inference API"
$ git push -u origin feature/ml-prediction-engine

```

5.2 Merging via Pull Request

Once the feature branch passes local testing, it is pushed to the remote and merged into development through a pull request, which is reviewed before merging. This produces a merge commit that preserves the full history of the feature branch.

```

$ git checkout development
$ git pull origin development
$ git merge --no-ff feature/ml-prediction-engine
Merge made by the 'ort' strategy.
 ml-service/app.py          | 84 +++++
 ml-service/requirements.txt |  6 ++
 2 files changed, 90 insertions(+)
$ git push origin development

```

5.3 Conflict Resolution

A merge conflict arose when both the frontend and backend feature branches independently modified the shared docker-compose.yml file to add their own service definitions. The conflict was resolved by manually combining both service blocks into a single, correct file before completing the merge.

```

$ git merge feature/frontend-dashboard
Auto-merging docker-compose.yml
CONFLICT (content): Merge conflict in docker-compose.yml
Automatic merge failed; fix conflicts and then commit the result.
# Conflict markers were opened in the editor, both service blocks
# were kept and correctly indented under 'services:'
$ git add docker-compose.yml
$ git commit -m "merge: resolve docker-compose service conflict"
$ git push origin development

```

5.4 Repository Synchronization

Before starting any new work, and before opening a pull request, the local development branch is synchronized with the remote to minimize the chance of large, hard-to-resolve conflicts later.


```
$ git fetch origin
$ git pull origin development --rebase
$ git status
On branch development
Your branch is up to date with 'origin/development'.
```

[Insert Screenshot Here]

Git branch graph showing feature, development, and main branches

6. COMMIT HISTORY ANALYSIS

6.1 Sample Commit History

Table 6.1: Representative commit log (git log --oneline)

Hash	Message
a3f9c1e	chore: initial commit with gitignore and readme
b71de44	feat(db): add MySQL schema for athletes, training, injuries
d4e8a27	feat(ml): add injury risk prediction model and inference API
f0c1d88	merge: resolve docker-compose service conflict
1a2b3c4	feat(docker): add Dockerfiles for backend and ml-service
9g0h1i2	fix: correct MySQL connection string in application server
3j4k5l6	docs: update README with Docker Compose setup instructions

6.2 Commit Message Conventions

Commit messages follow the Conventional Commits format, type(scope): short description, which is a widely used good practice for version control. The type prefix (feat, fix, chore, docs, test, merge) immediately communicates the nature of a change without opening the diff, the optional scope (db, backend, ml, frontend, docker) identifies which module was touched, and the description is written in the imperative mood ('add', not 'added') to match Git's own generated messages.

This convention contributes directly to project maintenance in three ways: it makes git log human-scannable when auditing what changed and why; it allows automated tooling (such as changelog generators or semantic-versioning bots) to categorize commits without manual tagging; and it makes code review faster, since a reviewer can predict the shape of a diff from the message alone before opening it.

7. DOCKER ENVIRONMENT DESIGN

7.1 Why Docker Is Suitable for This Project

Athlete-Shield AI is composed of three services built on entirely different runtimes — a Node.js application server, a Python machine learning inference service, and a MySQL database — plus a frontend build step. Without containerization, every developer and every deployment target would need to independently install and version-match Node.js, Python, the correct ML libraries, and MySQL, which is fragile and error-prone. Docker solves this by packaging each service together with its exact runtime and dependencies into a portable image that behaves identically everywhere it runs, directly satisfying the CO3 outcome of building full-stack applications with an emphasis on containerization.

7.2 Application Components Requiring Containerization

Table 7.1: Components selected for containerization

Component	Base Technology	Reason for Containerization
frontend	Node.js build + Nginx	Isolates the JS toolchain and serves a static production build
backend (application server)	Node.js + Express	Isolates business-logic dependencies and exposes a stable API port
ml-service (prediction engine)	Python + scikit-learn/Flask	Isolates heavy ML dependencies from the rest of the stack
database	MySQL 8	Provides a reproducible, disposable database instance for dev/test

8. DOCKERFILE DEVELOPMENT

8.1 Dockerfile — Application Server (backend/Dockerfile)

```
# ---- Base image with Node.js runtime ----
FROM node:20-alpine
# ---- Set working directory inside the container ----
WORKDIR /usr/src/app
# ---- Copy dependency manifests first (layer-cache friendly) ----
COPY package*.json ./
# ---- Install only production dependencies ----
RUN npm install --omit=dev
# ---- Copy the rest of the application source ----
COPY . .
# ---- Document the port the app server listens on ----
EXPOSE 5000
# ---- Default command to run the server ----
CMD ["node", "src/index.js"]
```

8.2 Dockerfile — Injury Prediction Engine (ml-service/Dockerfile)

```
# ---- Base image with Python runtime ----
FROM python:3.11-slim
WORKDIR /app
# ---- Install system dependencies needed by ML libraries ----
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential && rm -rf /var/lib/apt/lists/*
# ---- Install Python dependencies ----
```

```

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
# ---- Copy application and trained model artifacts ----
COPY . .
EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]

```

8.3 Instruction-by-Instruction Explanation

Table 8.1: Purpose of each Dockerfile instruction

Instruction	Purpose
FROM	Selects the minimal base image (Alpine/Slim variants) to keep final image size small
WORKDIR	Sets the working directory so subsequent commands run in a consistent location
COPY (manifests first)	Copies only dependency files first so Docker can cache the install layer
RUN npm install / pip install	Installs dependencies inside the image, baked into a cached layer
COPY . .	Copies the remaining application source code into the image
EXPOSE	Documents which port the container listens on (does not itself publish the port)
CMD	Defines the default command executed when a container starts from the image

9. DOCKER COMPOSE DESIGN

9.1 docker-compose.yml

```

version: "3.9"
services:
  frontend:
    build: ./frontend
    ports:
      - "3000:80"
    depends_on:
      - backend
    networks:
      - athlete-net
  backend:
    build: ./backend
    ports:
      - "5000:5000"
    env_file:
      - .env
    environment:
      - ML_SERVICE_URL=http://ml-service:8000
    depends_on:
      - db
      - ml-service
    networks:
      - athlete-net

```

```

ml-service:
  build: ./ml-service
  ports:
    - "8000:8000"
  networks:
    - athlete-net
db:
  image: mysql:8
  restart: unless-stopped
  environment:
    - MYSQL_ROOT_PASSWORD=${DB_ROOT_PASSWORD}
    - MYSQL_DATABASE=athlete_shield
  volumes:
    - db-data:/var/lib/mysql
    - ./database/init.sql:/docker-entrypoint-initdb.d/init.sql
  networks:
    - athlete-net
volumes:
  db-data:
networks:
  athlete-net:
    driver: bridge

```

9.2 Explanation of Services, Networking, Volumes, and Dependencies

- **Services:** four services are defined — frontend, backend, ml-service, and db — each built from its own Dockerfile except db, which uses the official mysql:8 image directly.
- **Networking:** all services join a single custom bridge network (athlete-net), which allows them to reach each other by service name (for example, the backend calls the ML engine at http://ml-service:8000) instead of hard-coded IP addresses.
- **Volumes:** a named volume (db-data) persists MySQL's data directory across container restarts and rebuilds, so athlete records are not lost when the db container is recreated; init.sql is bind-mounted so the schema is created automatically on first startup.
- **Environment Variables:** sensitive values such as MYSQL_ROOT_PASSWORD are supplied through a .env file (excluded from Git via .gitignore) rather than hard-coded into the compose file, keeping credentials out of version control.
- **Dependencies:** depends_on ensures the backend only starts after the database and ML service containers have been created, and the frontend only starts after the backend, giving the stack a correct, deterministic startup order.

10. CONTAINER DEPLOYMENT AND TESTING

10.1 Building and Running the Stack

```

$ docker-compose up --build -d
[+] Building 42.1s (28/28) FINISHED
[+] Running 5/5
 - Network athlete-shield-ai_athlete-net    Created
 - Volume  athlete-shield-ai_db-data        Created
 - Container athlete-shield-ai-db-1         Started
 - Container athlete-shield-ai-ml-service-1 Started
 - Container athlete-shield-ai-backend-1    Started
 - Container athlete-shield-ai-frontend-1   Started
$ docker-compose ps

```

NAME	STATUS	PORTS
athlete-shield-ai-frontend-1	Up 10 seconds	0.0.0.0:3000->80/tcp

athlete-shield-ai-backend-1	Up 10 seconds	0.0.0.0:5000->5000/tcp
athlete-shield-ai-ml-service-1	Up 10 seconds	0.0.0.0:8000->8000/tcp
athlete-shield-ai-db-1	Up 10 seconds	0.0.0.0:3306->3306/tcp

10.2 Testing Procedure and Evidence

Each service was verified independently before verifying the full end-to-end flow. The ML service's health endpoint and the backend's athlete-listing endpoint were exercised directly with curl, and the complete risk-prediction flow was exercised through the frontend dashboard.

```
$ curl http://localhost:8000/health
{"status": "ok", "model_loaded": true}
$ curl http://localhost:5000/api/athletes
[{"id":1,"name":"A. Kumar","sport":"Sprint"}, ...]
$ curl -X POST http://localhost:5000/api/predict -H "Content-Type: application/json" \
-d '{"athleteId":1,"trainingLoad":78,"restHours":5}'
{"athleteId":1,"riskLevel":"High","riskScore":0.82,
"recommendation":"Reduce training load and increase recovery time"}
```

[Insert Screenshot Here]
docker-compose ps showing all four containers running

[Insert Screenshot Here]
Browser view of the coach dashboard showing a generated risk score

All four containers reached and remained in the Up state, the ML service's health check confirmed the trained model was loaded successfully, and a sample high training-load, low rest-hours input correctly produced a High risk score with an accompanying recommendation, confirming that the containerized stack reproduces the same behaviour as the architecture designed in the earlier system architecture assignment.

11. BENEFITS OF GIT AND DOCKER FOR THIS PROJECT

Table 11.1: Benefits mapped to project-specific outcomes

Benefit	How It Applies to Athlete-Shield AI
Collaboration	Frontend, backend, and ML contributors work in parallel on separate feature branches without blocking each other
Version Management	Every change to the risk-prediction logic is traceable, reviewable, and revertible through commit history
Portability	The Node.js, Python, and MySQL services run identically on any machine with Docker installed, regardless of the host's installed software

Benefit	How It Applies to Athlete-Shield AI
Deployment	docker-compose up --build reproduces the entire multi-service stack with a single command, removing manual setup steps
Scalability	Individual services (e.g., ml-service) can be scaled or redeployed independently as usage grows, without touching the others
Maintainability	Clear branch structure and conventional commit messages make it straightforward to locate when and why a change was introduced

12. CHALLENGES AND IMPROVEMENTS

12.1 Challenges Encountered

- Merge conflicts in shared configuration files: both the frontend and backend feature branches modified docker-compose.yml simultaneously, requiring manual conflict resolution as shown in Section 5.3.
- Container networking confusion: early attempts had the backend calling the ML service via localhost, which fails inside Docker's isolated network namespace; this was resolved by switching to the service name (ml-service) as the hostname.
- Image size and build time: the ML service's dependencies (scikit-learn, pandas, numpy) initially produced a large image and slow rebuilds during development.
- Environment variable management: keeping database credentials out of version control while still making them available to every container required careful .env and .gitignore discipline.

12.2 Suggested Improvements and Future Enhancements

- Adopt a CI/CD pipeline (e.g., GitHub Actions) that automatically runs tests and builds Docker images on every push to development, catching integration issues before merge.
- Use multi-stage Docker builds for the ML service to discard build-time dependencies from the final image, reducing its size and improving deployment speed.
- Introduce a Docker layer cache in CI to significantly speed up repeated builds during active development.
- Add automated database migrations (rather than a single init.sql) so schema changes can be version-controlled and applied incrementally.
- Move from a single docker-compose stack to Kubernetes manifests for production, enabling independent auto-scaling of the ML service under heavy load.
- Enforce branch protection rules on main and development requiring passing CI checks and at least one review before merge.

CONCLUSION

This report has applied Git version control and Docker containerization to the Athlete-Shield AI Injury Risk Prediction System, covering the complete lifecycle from problem analysis and repository design through branching strategy, commit discipline, containerization of each service, Compose-based orchestration, and deployment testing. The Gitflow branching model kept parallel

frontend, backend, and machine-learning work isolated and reviewable, while Docker and Docker Compose ensured that the project's heterogeneous runtime requirements — Node.js, Python, and MySQL — could be built, tested, and deployed identically across every environment. Together, these practices directly support the CO3 outcome of building full-stack applications with containerization, version control, and collaborative development. All required deliverables are documented within this report: the Git repository and commit history (Section 6.1), source code layout (Section 2.1), Dockerfiles (Section 8), docker-compose.yml (Section 9.1), Git/Docker execution evidence (Sections 3, 5, 10), and the branch topology and system architecture diagrams (Figure 4.1).

REFERENCES

- [1] Git Documentation, "Pro Git Book," Software Freedom Conservancy. [Online]. Available: <https://git-scm.com/book>
- [2] Docker Inc., "Docker Documentation," Docker. [Online]. Available: <https://docs.docker.com>
- [3] Docker Inc., "Docker Compose File Reference," Docker. [Online]. Available: <https://docs.docker.com/compose/compose-file/>
- [4] V. Driessen, "A Successful Git Branching Model," nvie.com, 2010. [Online]. Available: <https://nvie.com/posts/a-successful-git-branching-model/>
- [5] Conventional Commits, "Conventional Commits v1.0.0." [Online]. Available: <https://www.conventionalcommits.org>
- [6] MySQL, "MySQL 8.0 Reference Manual," Oracle Corporation. [Online]. Available: <https://dev.mysql.com/doc/>